

Detecting Python Type Errors through Interprocedural Type-State Analysis

ANONYMOUS AUTHOR(S)

Python type errors depend on the state in which an operation executes. A function call can change the type of an object attribute or a global variable, making a previously valid operation incompatible with the updated state. Detecting such errors requires connecting input conditions to the state changes established across calls. We present an interprocedural type-state analysis for Python type-error detection. Its central model relates function entries to ordered operation requirements, state updates, and exits, preserving both conditional effects and intermediate failures. The analysis constructs these relations by sharing facts independent of unknown entry types and using a large language model to propose implicit type alternatives. At each call, it instantiates object identities and propagates the corresponding state changes. A conflict triggers selective expansion of the relevant callee type paths and contextual re-inference; revised relations then update dependent callers. The analysis separately reports runtime type errors and annotation-contract errors. We evaluate paired detection on xxx recent GitHub bug-fix records, diagnostic precision, component effects, and total analysis cost. The evaluation reports xxx paired runtime detections and xxx paired annotation-contract detections, with respective diagnostic precision of xxx% and xxx%.

Additional Key Words and Phrases: Python, type errors, type-state relations, interprocedural analysis, function summaries

1 Introduction

Python’s dynamic type system offers developers considerable flexibility and enables rapid development. However, it also allows type errors to remain undetected until runtime. These errors can interrupt program execution and cause application failures, compromising software reliability. Consequently, substantial research effort has been devoted to static type-error detection for Python, with the aim of identifying such errors before execution.

Existing approaches have made progress in static type-error detection for Python. Type checkers such as Mypy and Pyrely use type annotations to check assignments, function calls, and other operations against declared types, and have been adopted in major production codebases and open-source projects, including Dropbox, Instagram, PyTorch, and JAX [2, 7]. Automatic type inference further improves detection. For example, Pyinder detected 50% (34/68) of real-world type errors in its benchmark [13]. However, these approaches rely on static type models that do not fully capture how types vary with execution conditions or change through function calls. As a result, they can produce false alarms or miss type errors. In Pyinder’s evaluation, such limitations accounted for 31% of the confirmed false alarms in its inspected sample [13]. This gap between static type models and Python’s dynamic typing behavior limits the effectiveness of existing type-error detection methods.

To address this gap, we aim to improve Python type-error detection by tracking type changes across function calls. A function’s type requirements concern not only its arguments but also the global variables, instance attributes, and other values it accesses. We refer to their collective type information at a given program point as the *type-state*. Our approach captures how a function transforms this state under different execution conditions and uses these transformations to check subsequent operations. To achieve this goal, we need to address three key challenges.

Challenge 1: Modeling type-state changes across function calls. A function can change the caller’s type-state not only through its return value but also by updating global variables or attributes of mutable objects. These updates depend on the function’s execution path, so calls to the same function can establish different type-states. Conventional type annotations describe parameter,

return, and attribute types, but do not specify how a call transforms the incoming type-state under different execution conditions. We therefore need a new model to express these conditional type-state changes and their effects on subsequent operations.

Challenge 2: Extracting conditional type-state changes from code. Program execution involves both explicit and implicit paths. Explicit paths arise from branch statements, while implicit paths arise from Python operations whose behavior depends on the types involved. For example, addition can perform arithmetic, concatenate strings, or invoke a user-defined method, with different type requirements and effects in each case. Analyzing these paths across function calls further requires considering combinations of caller and callee paths, leading to path explosion. Accounting for implicit paths and managing path explosion therefore make it challenging to extract conditional type-state changes from code.

Challenge 3: Detecting type errors using conditional type-state changes. Due to limitations in analysis precision, the extracted type-state transition relations may contain inaccuracies, which can cause valid operations to be flagged as type errors. A detected type conflict therefore does not necessarily indicate an error in the program. Distinguishing a real error from an analysis inaccuracy requires examining how the conflicting state arises under the relevant calling conditions. Moreover, an inaccurate type-state change can propagate through multiple calls, affecting error detection in other functions. The challenge is therefore to distinguish type errors from false alarms and correct analysis inaccuracies.

To address these challenges, we propose PyTT(Python Type Tracker), a Python type-error detector that models and tracks conditional type-state transitions across function calls. To address **Challenge 1**, we introduce a *path-sensitive type contract* (PSTC) to represent these transitions, capturing a function’s type requirements and updates along different execution paths. To address **Challenge 2**, PyTT uses static analysis to identify explicit paths and a large language model (LLM) to infer implicit paths. It further uses static analysis to simplify code whose type behavior is independent of the calling context, reducing repeated analysis across paths to mitigate path explosion. To address **Challenge 3**, PyTT uses PSTCs to trace detected conflicts across function boundaries. It analyzes the resulting traces to distinguish type errors from false alarms. For false alarms, PyTT corrects the inaccuracies in the corresponding contracts and updates the affected analysis results.

We evaluate PyTT on xxx real-world type-error fixes from xxx Python repositories, measuring detection effectiveness, diagnostic precision, and analysis cost. PyTT detects xxx runtime type errors and xxx annotation-contract errors, with precision of xxx% and xxx%, respectively. Each detection is checked against both the buggy and fixed revisions. Ablation experiments examine the contributions of conditional type-state modeling, explicit and implicit path analysis, and conflict analysis and correction. We further evaluate PyTT on xxx independent repositories, where it identifies xxx previously unknown type errors.

The contributions are as follows:

- We introduce the path-sensitive type contract (PSTC), a model that captures conditional type-state transitions across function calls for Python type-error detection.
- We propose PyTT, a type-error detection method that combines static analysis and LLM-based analysis to extract these transitions. PyTT analyzes conflict traces across functions to distinguish type errors from false alarms and correct analysis inaccuracies.
- We evaluate PyTT on real-world type-error fixes and independent Python repositories, assessing detection effectiveness, diagnostic precision, and analysis cost, and examining the contributions of its key components through ablation experiments.

2 Background and Motivating Example

In this section, we explain how dynamic typing and duck typing in Python affect type-error detection. We then introduce Hoare triples and explain how their preconditions and postconditions can summarize function behavior.

2.1 Dynamic Typing and Duck Typing in Python

A type-error detector must determine which types an operation's operands can have and check whether the operation supports those types [13]. Python's dynamic typing and duck typing complicate these tasks. With dynamic typing, the values used by an operation may have different types on different executions. Under duck typing, objects of different classes can support the same operation. To check compatibility, a detector must examine whether the operands meet the requirements of the methods or protocols used by that operation. Moreover, the same operation may invoke different implementations for different operand types. These implementations may return values of different types, affecting the types used by subsequent operations.

More specifically, **dynamic typing** allows both local and global variables in Python to be bound to objects of different types during program execution [15, 16]. Consequently, an operand's type depends on the assignments and updates along the execution path to the operation. Function calls can also change the values that a caller uses. A caller may assign a function's return value to a variable. A callee may also update a global variable or an attribute of an object shared with its caller. When the caller next reads the updated variable or attribute, it may obtain a value of a different type, even if the callee did not return that value. To determine the types used by subsequent operations, a detector must therefore track assignments and updates both within functions and across calls.

In Python, code written in the **duck typing** style uses objects through the methods and attributes they support, without requiring them to belong to a particular class [17]. For an operation that calls a method on an operand, a detector must check that the object supports that method and that the call satisfies the method's requirements. Beyond checking whether an operation accepts its operands, a detector must account for what its implementation returns or updates. Python's dispatch rules may select different implementations for different operand types. These implementations may return values of different types or update state that subsequent operations read. The detector must track these effects because later operations use the resulting values and updated state.

To check subsequent operations precisely, a detector can relate the types and state before each operation to its result and the state after it. For a function call, this relation connects the argument types and the types of shared values before the call to the return type and the types of updated values after it. The shared values include global variables and object attributes that the function reads or updates. The same function may return or update values of different types under different calling conditions or along different execution paths. If the detector records only the possible resulting types, it loses the conditions under which each type arises. It may then consider types that cannot occur in the current calling context when checking a later operation. Keeping each result associated with its conditions allows the detector to use the type information relevant to that call.

2.2 Hoare Triples as Function Summaries

To check operations after a call, a detector must account for the values returned or updated by the callee. A function summary describes the callee's relevant behavior so that the detector can reuse it at call sites. For type-error detection, a summary can relate conditions on the arguments and shared state before the call to type facts that hold after it returns normally. Hoare triples provide a standard way to express this relationship between conditions before and after execution.

Hoare logic describes how executing a program changes its state [5, 14]. A Hoare triple has the form

$$\{P\} C \{Q\}, \quad (1)$$

where C is a command, and P and Q are assertions over program states, called the precondition and postcondition, respectively. An assertion describes a property of a state, such as the type of a value held in a variable. We use a partial-correctness interpretation that describes what holds on normal termination. A valid triple states that if P holds before C executes and C terminates normally, then Q holds afterward. In other words, P gives the assumptions under which executing C establishes Q on normal termination. Under this interpretation, the triple does not guarantee that C terminates or executes without raising a `TypeError`.

When a Hoare triple is used as a function summary, its precondition can express assumptions about the arguments and shared state, while its postcondition can describe the return value and properties of the shared state after the call. To apply the summary at a call site, the detector matches the callee's parameters with the caller's arguments and interprets references to shared objects in the caller's state. This step instantiates the summary for the call. If the calling state satisfies the instantiated precondition, the postcondition gives facts that hold when the call returns normally. The detector can use these facts to check subsequent operations. If the calling state does not satisfy the precondition, the detector cannot rely on this guarantee. That alone does not establish a type error. To determine whether an error occurs, the detector must examine the relevant operations and their exceptional behavior.

Hoare triples also support sequential composition. If $\{P\} C_1 \{R\}$ and $\{R\} C_2 \{Q\}$ are valid, then $\{P\} C_1; C_2 \{Q\}$ is valid. Here, R is an intermediate assertion. Starting from a state satisfying P , C_1 establishes R on normal termination. This assertion then serves as the precondition for the second triple. Thus, the properties established by one command provide the assumptions for reasoning about the next command.

For Python type-error detection, a detector can use a function summary to update the type facts it uses after a call. If a function updates an attribute of a shared object, its postcondition can describe the type of the value held in that attribute on normal return. The caller still accesses the same object, but the attribute now holds the updated value. A type fact about its previous contents may therefore no longer apply. The detector must check subsequent operations using type facts that account for the update. When the calling state satisfies the summary's precondition, its postcondition can provide these facts.

2.3 Motivating Example

Figure 1 presents a message-processing example that illustrates the need to track type-state changes across function calls. The function `make_line` calls `encode_message` to encode a message body before appending a newline. The body is declared as `str | bytes`. For a string body, `encode_message` invokes the supplied encoder and stores its result in the same message object. `TextEncoder` returns the string unchanged, whereas `Utf8Encoder` converts it to bytes.

As shown in the figure, `Mypy`, `Pyre`, and `Pyright` do not report the concatenation error, whereas `PyTT` detects it. Analyzing this example reveals two related requirements for type-error detection. First, the analyzer must propagate changes to the shared message body across the call so that the subsequent operation is checked against its updated type. Second, distinguishing the failing UTF-8 call from the safe text-encoder call requires retaining the conditions under which each update occurs. We examine these requirements below and explain how `PyTT` addresses them through conditional type-state summaries.

Function with Type Error

```
def make_line(msg:Message):
    if not isinstance(msg.body, str):
        raise ValueError("...")
    1 #msg.body: str
    encode_message(msg, Utf8Encoder())
    3 #msg.body: bytes
    print(msg.body + '\n')
    #TypeError: can't concat str to bytes
```

Function with Complex Type Transformation

```
def encode_message(msg: Message,
                  encoder:TextEncoder|Utf8Encoder):
    if isinstance(msg.body, str):
        msg.body = encoder.encode(msg.body)
    2 #encoder: TextEncoder => msg.body: str
    #encoder: Utf8Encoder => msg.body: bytes
```

Related Classes

```
class Message:
    body: str | bytes

class TextEncoder:
    def encode(self, text: str) -> str:
        return text

class Utf8Encoder:
    def encode(self, text: str) -> bytes:
        return text.encode("utf-8")
```

Detection Results

```
Mypy, Pyrefly, Pyright: No diagnostic
PyTT: Type error detected
```

Fig. 1. Motivating example of interprocedural type-state changes and type-error detection results.

(1) Interprocedural Type-State Propagation. In `make_line`, the guard raises `ValueError` when `msg.body` is not a string. Thus, the body has type `str` at point 1. The subsequent call passes `msg` and a `Utf8Encoder` instance to `encode_message`, which writes the encoding result into `msg.body`. Because both functions access the same message object, the body has type `bytes` at point 3 after the call returns normally. The expression `msg.body + '\n'` then raises `TypeError` before `print` is invoked, and the exception escapes `make_line`.

Both the original string and the updated bytes satisfy the field’s union annotation, but the updated body does not support concatenation with a string. Detecting this error requires propagating the field update from `encode_message` to its caller. The function’s return type, `None`, does not describe this update. If the analyzer retains the string type established at point 1, it can miss the error at point 3.

(2) Conditional Type-State Modeling. Propagating a field update also requires determining which type the call establishes. In this example, that type depends on the encoder argument. Replacing `Utf8Encoder` with `TextEncoder` leaves a string body and makes the concatenation valid. An analyzer that assigns `str | bytes` to the body after either call can flag the potential bytes/string conflict. However, retaining the bytes alternative for `TextEncoder` can also cause a false alarm. Distinguishing these calls therefore requires preserving the relation between the encoder argument and the resulting field type under the incoming string condition.

PyTT addresses these requirements using path-sensitive type contracts (PSTCs), which record field updates together with their calling conditions and propagate the applicable updates to callers. At point 2, the contract for `encode_message` records two cases for an incoming string body. On normal return, the `TextEncoder` case establishes a string body, and the `Utf8Encoder` case establishes a bytes body. For a non-string input, the function leaves the body unchanged.

At the illustrated call, PyTT uses the string type established by the guard and the `Utf8Encoder` argument to select the second case. It maps the callee’s message parameter to the caller’s object and updates the type of `msg.body` accordingly. The subsequent concatenation is therefore checked with a bytes body, revealing the type error.

3 Approach

This section presents how PyTT detects Python type errors by tracking conditional type-state changes across function calls. We first explain the overall workflow and the function contracts used to represent these changes. We then describe how PyTT extracts the contracts from code, uses them to check operations, and investigates the resulting conflicts.

3.1 Overview

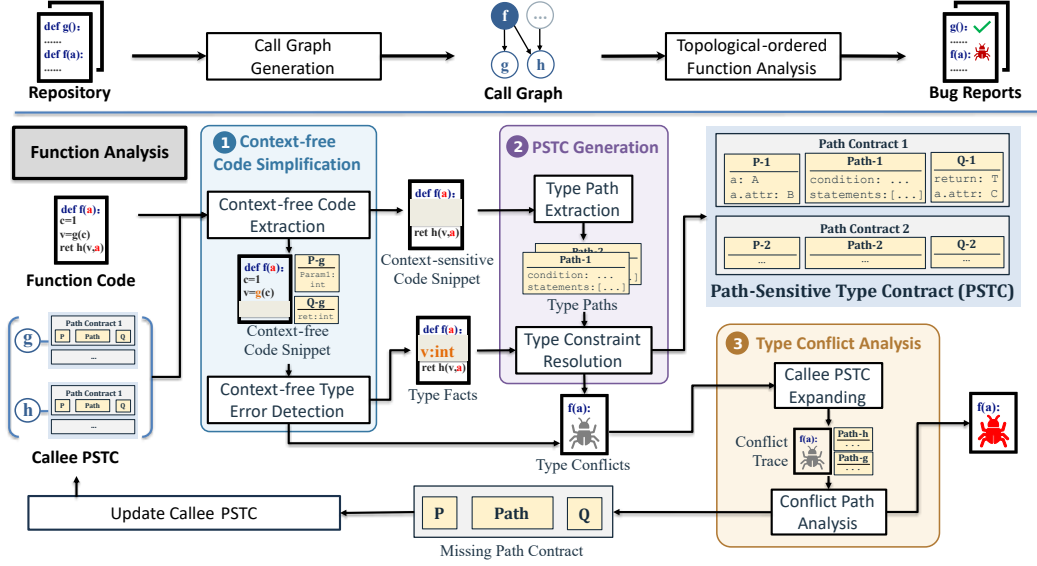


Fig. 2. Overview of PyTT's workflow

To check an operation after a function call, PyTT needs the types established by that call under the current calling conditions. It records this information in a *path-sensitive type contract* (PSTC), which relates a function's entry conditions to its return value and shared-state updates along different paths. A caller can use the relevant contract cases to update its type-state and check subsequent operations.

Figure 2 shows how PyTT constructs and uses these contracts for a Python repository. PyTT builds a call graph and gives priority to analyzing callees, whose contracts then provide information for analyzing callers. For each function, it performs three phases.

(1) Context-free Code Simplification. Analyzing a function under different calling conditions can repeat work on computations whose type behavior is already determined. PyTT first identifies and checks these computations using static analysis. It then reuses their type facts when analyzing the remaining code, reducing the work needed to consider different paths.

(2) PSTC Generation. The remaining code can produce different types under different entry conditions. PyTT extracts these conditional transitions by identifying explicit paths with static analysis and inferring implicit paths with an LLM. It combines the requirements and effects along each path with the facts from Phase I and available callee contracts. The resulting PSTC records the transitions used to check operations in callers.

(3) Type Conflict Analysis. A conflict found using an inferred contract may reflect either a program error or an inaccuracy in that contract. To distinguish them, PyTT traces the conflicting

type facts to the relevant caller and callee paths and analyzes those paths under the calling conditions. It reports errors supported by this analysis and corrects contracts responsible for false alarms. Cases with insufficient evidence remain unresolved.

These phases are connected through the contracts they construct and revise. A callee contract is used to analyze its callers; a correction to that contract can therefore change their analysis results. PyTT repeats the affected analyses until no pending work remains or the configured budget is reached. It then reports confirmed type errors and records undecided cases as unresolved. The PSTC model addresses Challenge 1 in the introduction, its extraction in Phases I and II addresses Challenge 2, and conflict analysis in Phase III addresses Challenge 3.

3.2 Design of Path-Sensitive Type Contracts

The motivating example establishes two requirements for a function summary. It must describe changes to values shared with the caller, and it must retain the conditions under which each change occurs. For `encode_message`, knowing that the function returns `None` does not tell the caller what happened to the message body. Recording that the body can become either a string or bytes still leaves the caller unable to determine which result applies to its encoder. PyTT therefore needs a summary that connects the incoming type-state and execution conditions to the types established by the call.

3.2.1 Representing Calling Conditions and Updates. To express changes beyond the return value, the summary must refer to the variables and attributes read or updated by the function. PyTT represents their type information at a program point as a *type-state*

$$S : L \rightarrow \mathcal{T},$$

where L contains the tracked locations and $\mathcal{T}$ is the domain of type information, including sets of possible types. A location can be a parameter, a local or global variable, or a class or instance attribute. A distinguished result location records the return value on normal exit. Thus, $S(\ell)$ describes the type information for the value held at location ℓ . An assignment to an attribute changes the facts for that attribute's contents. If the caller and callee refer to the same object, this update also changes the facts used when the caller next reads the attribute.

The summary must also specify when these post-call facts apply. PyTT expresses this information using *state assertions*, which describe sets of states through type facts, branch conditions, and relevant object-identity constraints. An entry assertion can require a string message body and a `Utf8Encoder` argument, while an exit assertion can state that the body holds bytes after normal return. Associating these assertions keeps the resulting field type connected to the calling condition that establishes it. Branch conditions remain relevant because identical entry types need not imply identical execution paths.

3.2.2 Connecting Assertions through Execution Paths. Entry and exit assertions describe the conditional change needed by a caller. To check that change and investigate a conflict, PyTT also needs to retain the operations that produce it. For example, an exit assertion that the message body holds bytes does not show which call produced the bytes or which assignment stored them. PyTT therefore associates each pair of assertions with a path description. This description records guards, operation requirements, calls, and updates in execution order. It includes explicit branch choices and implicit choices of behavior that depend on operand types.

These three components form a path contract (p, m, q) , where p is the entry assertion, m is the path description, and q is the exit assertion. A function can have several such cases, collected in its PSTC

$$PSTC(f) \subseteq \mathcal{A} \times \mathcal{M}_f \times \mathcal{A},$$

where $\mathcal{A}$ is the set of state assertions and $\mathcal{M}_f$ is the set of path descriptions for f . Each case keeps a particular path associated with its entry conditions and post-call facts. The PSTC thus represents the information required to transfer type facts across a call while retaining the behavior behind those facts.

A path contract specializes the Hoare-triple structure introduced in the background. A correct case states that if execution starts in a state satisfying p , follows m , and returns normally, then q holds afterward. The caller can use these facts under the corresponding conditions. This partial-correctness interpretation describes normal returns; it does not guarantee that execution reaches one without a type error. PyTT checks the operations along the path and tracks their exceptional continuations separately. Moreover, the cases extracted by the analysis may contain inaccuracies, which Phase III investigates when conflicts arise.

For `encode_message`, one case starts with a string body and a `TextEncoder` argument. Its path calls the encoder and assigns the returned string to the body, establishing a string body on normal return. A second case associates a `Utf8Encoder` argument with a path that writes bytes to the body. These two string-input cases both return `None` but establish different field types. A third case covers a non-string input and leaves the body unchanged. Together, the cases show that the contract must preserve both the incoming condition and the update established on its path. The next two phases explain how PyTT obtains this information without repeating the same analysis for every calling condition.

3.3 Phase I: Context-free Code Simplification

Constructing the contract defined above requires examining behavior under different entry conditions and execution paths. When a path contains calls, its analysis also depends on the relevant paths of the callees, increasing the number of combinations to consider. Reanalyzing the entire function for each combination would repeat computations whose type behavior does not depend on the unknown entry state. PyTT first separates these computations and analyzes them once, so that subsequent path analysis can reuse their results.

3.3.1 Identifying Computations for Shared Analysis. The first step is to determine which computations can be analyzed in this way. We call a computation *context-free* when local semantics and available callee PSTCs determine its type behavior without resolving the unknown entry type-state. Here, a computation consists of an operation and the definitions, guards, shared-state accesses, and callee effects on which its type behavior depends. Starting from the operation, PyTT follows these dependencies backward. It places the computation in C_I only when every required type and effect can be resolved from local semantics, already established facts, or applicable callee PSTCs. If a dependency reaches an unknown entry value, an unresolved shared-state read, or an uncovered callee behavior, the affected computation remains in C_D , the *Context-sensitive Code Snippet* in the workflow figure.

For $f(a)$ in Figure 2, the assignment $c = 1$ provides a known integer argument to g . Together with the depicted PSTC of g , this determines that the result v is an integer without knowing the type of a . The computation can therefore be analyzed in C_I . The call $h(v, a)$ still depends on a and remains in C_D . This classification also considers shared-state dependencies of a call; known argument types alone do not establish that a call is context-free. For example, a call also remains in C_D when its callee may read or update a shared location whose incoming contents are unresolved.

3.3.2 Supplying Type Facts to the Remaining Code. Separating C_I is useful only if its results remain available where the rest of the function uses them. PyTT checks C_I using local operation semantics and callee PSTCs, and supplies the inferred type facts to their subsequent uses in C_D . In the figure, each analysis of $h(v, a)$ can start with the known integer type of v ; it does not need to infer $g(c)$

again for each possible type of a . A conflict encountered while checking C_I is passed to Phase III, just as a conflict in the remaining code would be.

These facts must remain tied to the execution that establishes them. A fact produced by an operation applies after that operation succeeds, and an intervening write can change the contents of the location it describes. PyTT therefore retains each fact's condition, value or location version, and supporting source or contract. It also preserves the source positions and control dependencies of C_I and C_D , so that their checks and updates can later be assembled in execution order. Branches, shared-state writes, and exceptional continuations in C_I remain part of the summarized behavior. Phase I thus provides reusable facts for path analysis without discarding the conditions of their use. If an intervening operation may overwrite a location and PyTT cannot identify a disjoint target, it invalidates the reusable fact rather than carrying that fact into C_D .

3.4 Phase II: PSTC Generation

Phase I determines the type facts that can be shared across calling conditions. It does not yet determine the behavior of C_D , such as the result of $h(v, a)$ for different types of a . To construct a PSTC, PyTT must now associate the remaining behavior with the conditions under which it occurs. It does this in two steps. Static analysis first identifies the explicit paths through the source code. The LLM then infers the type-dependent behavior within those paths, allowing PyTT to construct their entry and exit assertions.

3.4.1 Separating Explicit Execution Paths. Different branches can perform different updates, so combining their results before recording their conditions would lose information needed by the contract. PyTT uses static analysis to extract explicit paths through C_D within the configured bounds. Branch outcomes, exceptional successors, and bounded loop or recursive unfoldings create explicit path alternatives. Each path retains its guards and operations in source order, along with the facts from Phase I available at each use. These are the initial *Type Paths* in Figure 2. When a bound stops exploration, PyTT records the remaining frontier as uncovered; it does not treat the truncated path as a complete contract case.

In `encode_message`, the body test separates the path that calls the encoder and assigns its result from the path that leaves the body unchanged. This separation identifies where an update occurs and the body condition under which it occurs. It does not yet determine the type of the value returned by the encoder on the updating path.

3.4.2 Resolving Implicit Paths. The remaining uncertainty arises because a source operation can select different behavior for different operand types. The encoder call uses the implementation provided by its argument; similarly, $x + y$ can perform arithmetic, concatenate strings, or invoke a user-defined method. Enumerating source branches alone does not separate these implicit paths. Their requirements and effects must be determined to obtain the type-state changes along an explicit path.

PyTT uses an LLM to propose these paths one explicit path at a time. The input identifies the source operations whose behavior remains unresolved and supplies the facts established in Phase I, relevant repository type information, and applicable callee PSTCs. The requested output separates the operand-type conditions for each possible behavior from that behavior's operation requirements, result types, and shared-state effects. The LLM output is therefore a set of candidate implicit paths, not an independent error decision. PyTT attaches each proposal to the source operations and known facts from which it was inferred so that later composition and conflict analysis can inspect its support.

PyTT combines these results with the ordered checks and updates from C_I and C_D to form cases (p, m, q) . For the updating path of `encode_message`, the return type of each encoder determines

the type stored by the subsequent assignment. Keeping the encoder condition with that assignment yields a string-body case for `TextEncoder` and a bytes-body case for `Utf8Encoder`. Together with the unchanged-body path, these cases form the helper's PSTC. Each case retains its source locations, inference context, and supporting dependencies, so that the analysis can revisit how its facts were obtained if they later cause a conflict. If the available proposals do not cover an operand-type alternative represented by the current entry state, PyTT requests supplementary analysis when budget remains and otherwise retains that alternative as uncovered. A missing proposal alone does not justify either a safe result or an error report.

3.4.3 Applying Contracts to Check Caller Operations. The constructed PSTC describes the callee in terms of its parameters and the shared locations it accesses. To use it for a particular call, PyTT must first relate those references to the caller's values and objects. After evaluating the call target and arguments in source order and checking argument binding, PyTT constructs a substitution θ for this correspondence. For example, the helper's message parameter is mapped to the message object passed by `make_line`. This instantiation allows an update expressed through the callee's parameter to affect the same field that the caller later reads.

If two arguments refer to the same object, the substitution preserves their shared identity. Internal symbols and newly allocated objects are fresh for each call. These rules keep shared locations connected without merging objects that belong to different invocations. The same composition procedure is used when constructing caller contracts and when analyzing calls under given entry conditions.

After mapping the references, PyTT determines which cases are relevant to the call. Let A be the caller's abstract state and $\text{Facts}(A)$ its type facts and path conditions. For a case (p_i, m_i, q_i) , PyTT considers

$$\text{Facts}(A) \wedge \theta(p_i). \quad (2)$$

The conjunction restricts the calling states to those satisfying the case's entry assertion. An inconsistent case is excluded. A case that remains is analyzed under the combined condition, which is retained with its results. Several cases may apply to different parts of the calling state; consistency does not show that one case covers every possible execution. If the available cases leave states uncovered, PyTT requests further analysis or records those states as unresolved. Lack of a matching case alone does not establish a type error.

For each relevant case, PyTT follows the instantiated path in execution order to determine the state used by subsequent operations. Guards restrict the current condition, reads use the facts for the current location contents, and writes update those facts. On normal return, the exit assertion describes the resulting return value and shared state; it records the effects already processed along the path without applying them a second time.

A shared-state update must reflect which object is written. If the write definitely targets one location of a single object, PyTT replaces its previous type fact with the new one, a *strong update*. If several targets are possible, a *weak update* retains the possible old and new contents. Facts about locations known to be unaffected remain available. An unknown write invalidates precise facts for the region it may affect. Under this memory abstraction, later checks use only facts that have survived the call's possible write footprint; imprecise targets reduce precision instead of silently preserving an earlier fact.

As PyTT processes each operation, it checks the values used by that operation against the state established by the preceding steps. For an operation with requirement R and current facts F , it examines the success condition $F \wedge R$ and failure condition $F \wedge \neg R$. These describe the states in which the operands satisfy or violate the operation's requirement. If the failure condition is consistent with a represented calling condition, PyTT creates a candidate conflict and retains the source

location and the facts supporting that condition. If missing types or uncovered behavior prevent PyTT from establishing either alternative, the check remains unresolved. Normal continuation uses the success facts, while exceptional continuation carries the state established up to the failed operation.

At the motivating call, the caller's guard establishes a string body and the encoder argument is a `Utf8Encoder` instance. These facts select the case that writes bytes to the shared body. The following concatenation is consequently checked with a bytes operand and produces a bytes/string conflict. Replacing the encoder with `TextEncoder` selects the string-body case and satisfies the same operation requirement. The contract now provides the conditional update needed to distinguish the calls. Since that contract was inferred, however, a conflict obtained from it still requires the investigation described next.

3.5 Phase III: Type Conflict Analysis

The preceding phases check operations using type facts propagated from inferred contracts. If a contract loses a condition or assigns the wrong type to an update, a caller can receive an inaccurate fact and appear to perform an invalid operation. Inspecting only the operation cannot determine whether the program produces the conflicting value or the contract merely predicts it. PyTT therefore returns to the source behavior that established the disputed fact and analyzes it under the calling conditions. This phase first constructs a conflict trace and then uses that trace to decide whether to report an error or revise a contract.

3.5.1 Constructing a Conflict Trace. Reexamining every callee body would repeat analysis unrelated to the conflict. The path descriptions and supporting dependencies retained in the PSTCs allow PyTT to select the relevant behavior. Starting from the conflicting operation or annotation check, PyTT follows the conditions, values, and shared-object references that support its type facts. It uses the contributing callee cases to identify which paths need to be expanded in the call graph.

PyTT connects those callee paths to the caller fragments leading to the conflict. The resulting *conflict trace* retains guards, operation requirements, writes, and exceptional control flow, together with the call-site bindings established during composition. A write through a callee parameter can thus be followed to a later read through the caller's reference. The trace provides the source operations and calling conditions needed to reassess the disputed transition.

3.5.2 Distinguishing Errors from Contract Inaccuracies. Using the expanded trace, PyTT re-infers the type-state changes and checks the operation or annotation that produced the conflict. This step differs from initial contract construction in two ways. First, the candidate conflict fixes the calling conditions, object bindings, and disputed facts that the analysis must explain. Second, trace expansion replaces the contributing summarized segments with their relevant source paths instead of reconsidering every behavior of every callee. The re-inference therefore asks whether those source operations support the propagated fact and the failing requirement under the recorded calling condition.

For the motivating example, the trace connects the string guard to the encoder call, the field assignment, and the final concatenation. With `Utf8Encoder`, the encoder returns bytes and the helper stores them in the body. The concatenation then raises `TypeError`, which escapes `make_line`. The trace supports a runtime type-error report because it explains both how the bytes value arises and why the operation fails under that calling condition.

Suppose instead that an inaccurate contract predicts a bytes update for a `TextEncoder` call. Expanding the encoder's behavior reveals that it returns the string unchanged. The conflicting

bytes fact is then unsupported by the source behavior. PyTT corrects the contract under the text-encoder condition and dismisses the resulting false alarm. If the expanded analysis cannot establish an error or justify a correction, the candidate remains unresolved.

The re-inference result is not treated as a report merely because it repeats the initial prediction. A runtime report requires a continuous trace from a feasible entry condition through the value-producing updates to an escaping type-related failure. A contract correction instead requires the expanded source behavior to contradict or refine a specific condition, type, or object binding in the supporting case. When multiple expanded alternatives remain possible and support different outcomes, PyTT preserves them and leaves the candidate unresolved unless subsequent analysis separates their conditions.

The correction must be recorded in the contract so that subsequent callers receive the revised transition. PyTT adds or revises the relevant case, corresponding to the *Missing Path Contract* in Figure 2. Depending on the inaccuracy, this can restore a condition associated with an update, correct a type combination, distinguish object-identity cases, or supplement an uncovered entry case. A correction retains the calling conditions under which it was established and does not replace cases for other conditions. Other functions may already have used the previous case, so updating this contract also requires revisiting their dependent results, as described in Section 3.6.

3.5.3 Determining What to Report. The trace must support the kind of error being reported. For a *runtime type error*, it must reach an operation with incompatible operand types or unmet protocol requirements and show that the resulting type-related exception escapes the analyzed entry. For a repository entry, the trace includes the initialization and calls establishing that state. For an open entry, the report records the symbolic arguments and state assumptions under which the failure occurs. If a handler consumes the exception, PyTT records a handled failure. A corrected contract that removes the conflict leads to dismissal, while insufficient evidence leaves the candidate unresolved. Reports retain the trace, re-inference context and result, and exception disposition.

An *annotation-contract error* concerns whether a declaration agrees with the behavior of the code. PyTT checks whether declared input types meet the requirements of the operations using them, and whether returned or stored values satisfy their declared types. A conflict undergoes the same tracing and contextual re-inference as an operation conflict. A confirmed report identifies the declaration, the incompatible case, and the supporting behavior. Because the claim concerns the declaration's agreement with the code, it does not require a repository call that raises an exception. This separates the evidence needed for annotation conflicts from the runtime failures described above.

3.6 Interprocedural Refinement and Reuse

Phase III can correct a callee contract, but that correction alone does not update results already computed for its callers. PyTT records the contract dependencies of those results so that it can identify which analyses need to be repeated. When a PSTC changes, PyTT assigns it a new version, invalidates results that used its previous version, and re-enqueues the affected caller entries. Reanalysis can in turn change their contracts and propagate the correction to further callers. Algorithm 1 summarizes this feedback through a worklist of functions and their calling states.

Repeated analysis also creates opportunities to reuse work that remains applicable. The facts from Phase I, common path prefixes, and callee PSTCs can serve multiple cases and callers. Reusing a PSTC still requires instantiating its references and refreshing internal symbols for the current call. Reusing an already computed call result additionally requires matching its relevant entry state, object identities, call targets, analysis assumptions and settings, exploration bounds, and supporting contract versions. These conditions tie reuse to the facts on which the result depends.

Algorithm 1 PSTC propagation and conflict-driven refinement

Input: Repository P , fixed call graph G , entries E , budgets B
Output: Runtime reports, annotation-contract reports, and unresolved cases

- 1: Initialize PSTCs and a callee-first worklist W from G, E
- 2: **while** $W \neq \emptyset$ and analysis budget remains **do**
- 3: Remove a function entry (f, A) from W
- 4: Partition entry-independent computations and infer reusable facts
- 5: Extract bounded explicit paths and propose their implicit alternatives
- 6: Construct or supplement $PSTC(f)$; retain uncovered states
- 7: Compose PSTCs under A and collect candidate conflicts
- 8: **for** each candidate conflict c **do**
- 9: Trace supporting conditions, values, object identities, and callee cases
- 10: Expand relevant paths and re-infer behavior in the calling context
- 11: **if** the trace supports an escaping runtime failure **then**
- 12: Record a runtime report for c
- 13: **else if** the trace supports an annotation-behavior incompatibility **then**
- 14: Record an annotation-contract report for c
- 15: **else if** the trace supports a correction to a PSTC case **then**
- 16: Revise the PSTC and invalidate dependent results
- 17: Enqueue affected caller entries in W
- 18: **else**
- 19: Retain c as unresolved
- 20: **end if**
- 21: **end for**
- 22: **end while**
- 23: Retain uncovered states and undecided candidates as unresolved

The check includes dependencies of reads, checks, and updates; instances with unknown read or write locations are recomputed. A cached failure is reconsidered under the current entry's reachability and exception handling.

Reuse reduces repeated work but does not remove the growth in path combinations. PyTT bounds path exploration, loop and recursive expansion, and conflict refinement. Refinement of a candidate stops when its outcome is established, no new evidence is obtained, or its budget is exhausted. Unexamined continuations, uncovered states, and undecided conflicts remain unresolved. The complete cost includes shared fact inference, path construction, call-site composition, trace expansion, contextual re-inference, and dependency maintenance, including caller reanalysis after corrections. Section 4.3 evaluates this cost and examines reuse in comparison with direct function-body analysis.

4 Evaluation

We evaluate three questions: how effectively the analysis detects type errors (RQ1), how its mechanisms affect decisions and cost (RQ2), and what new errors it identifies in independent repositories (RQ3).

4.1 Experimental Setup

Benchmark. The historical benchmark contains xxx public GitHub bug-fix records from xxx Python repositories, collected over xxx. Each record identifies the buggy revision, its repaired

revision, the target operation or annotation, and the evidence linking the change to the error. Table 1 summarizes the corpus. The detector analyzes each revision separately using that revision’s source and available dependencies. Patch descriptions, cross-revision comparisons, and issue metadata are reserved for assessment. The corpus emphasizes recent public type errors. We assign runtime and annotation-contract labels separately, retaining a link when a record exhibits both categories.

A target’s category and entry scope determine its eligibility. Runtime targets satisfy the protocol-failure definition in Section 2; annotation targets concern the semantic compatibility of a declaration with behavior. For every tool, results retain the full benchmark denominator and the applicable-subset denominator. Unresolved targets, timeouts, and unsupported cases have separate counts.

Baselines. We compare Mypy, Pyright [9], Pyrefly [8], Pyinder, MOPSA on its supported subset, direct LLM detection, and annotation inference followed by a type checker. The checker baselines represent declared and inferred type checking; Pyinder addresses the same detection task [13]; MOPSA provides a comparison with relational Python analysis [10]. The annotation pipeline uses xxx followed by xxx and evaluates errors in the original program separately from inconsistencies introduced by generated annotations. Direct LLM detection uses xxx with the entry and repository context specified by xxx. Tool versions and configurations are xxx. Mypy uses `-check-untyped-defs` for checking unannotated function bodies [11].

Configuration. The analysis uses LLM xxx, decoding configuration xxx, and prompts xxx. Python and dependency versions are xxx, pyflow revision xxx, and hardware xxx. Entry selection and dependency models are xxx. The limits for explicit paths, implicit type alternatives, loop/recursive unfolding, conflict refinement, and wall-clock time are xxx. The aggregation rule for completion, resolution, and outcome counts across runs is xxx. Cost accounting includes all runs. These settings apply to the comparisons below.

Detection and precision. Paired detection requires a target report in the buggy revision and removal of the corresponding conflict in the fixed revision, with both analyses complete and the fixed target semantically resolved. For runtime errors, the buggy outcome must satisfy the entry and exception-escape criteria. For annotation-contract errors, the repaired declaration or behavior must resolve the confirmed incompatibility. A fixed target that becomes unreachable requires reachability evidence. Let N_k be the number of eligible records for category k and D_k its paired detections. We report D_k/N_k together with the full-pool rate. A record in both categories contributes once to each category and once to any unique-record aggregate.

Diagnostic precision is the fraction of reviewed reports confirmed as actual errors, including non-target errors. We report it separately for the two categories. Review follows the source, declared entry, repair or other behavioral evidence, and exception disposition; the adjudication procedure and reviewer agreement are xxx. Duplicate reports of the same root cause and category are grouped within a revision. Cross-category links are retained rather than counted as duplicate diagnoses.

Resolution and cost. Completion means the analysis process finishes within its resource limits; resolution means it establishes a target’s semantic disposition. We report both, together with unresolved, timed-out, and unsupported targets. Total cost includes call-graph construction, fact and relation extraction, composition, LLM requests, trace expansion, re-inference, and caller reanalysis. Measurements include wall-clock time, peak memory, requests, tokens, summary applications, and expanded source size. Medians and tail latencies are computed over the explicitly identified completed set; timeouts retain their consumed budget in total-cost accounting.

Table 1. Benchmark composition. Category counts can overlap; unique records are counted once.

Characteristic	Count or value
Historical repositories	xxx
Unique bug-fix records	xxx
Runtime type-error records	xxx
Annotation-contract error records	xxx
Records with both categories	xxx
Collection interval	xxx
Independent discovery repositories	xxx

4.2 RQ1: Detection Effectiveness

Table 2 reports paired detection and diagnostic precision by category. Our approach detects xxx runtime targets and xxx annotation contract targets, with precision of xxx% and xxx%, respectively. The corresponding full-pool paired rates are xxx% and xxx%.

Table 3 places detection beside resolution and cost. Our analysis completes xxx% of revision analyses and resolves xxx% of eligible targets. The unresolved, timed-out, and unsupported counts are xxx, xxx, and xxx. The median and 95th-percentile times are xxx and xxx seconds, and peak memory is xxx. Across all runs, the total number of LLM requests is xxx and token consumption is xxx.

We associate targets with the information needed to establish the checked state: input/return dependencies, condition-type relations, and attribute or global updates. These labels can overlap. For each differing outcome, we trace the operands and effects supporting the judgment. A difference is attributed to relation preservation when restoring the relation in a matched mechanism comparison changes the decision. The counts for these three dependence categories are xxx, xxx, and xxx; their respective paired detections are xxx, xxx, and xxx.

4.3 RQ2: Mechanism Effects and Cost

We compare controlled alternatives under the same entries, available repository facts, operation semantics, model configuration, and budgets. All variants use the same confirmation rules. Table 4 defines the interventions; Table 5 reports category-specific detections, unresolved targets, and total time.

State and condition preservation. Independent types, merged paths, and unknown effects isolate operand relations, branch correlations, and shared-state updates. Relative to the complete approach, their differences in paired runtime detection counts are xxx, xxx, and xxx; the corresponding differences in paired annotation-contract detection counts are xxx, xxx, and xxx. They change xxx, xxx, and xxx unresolved outcomes, respectively. Decision-level inspection links each change to the condition, value version, or object binding that reaches the target operation.

Construction and reuse. Full-body construction retains the same type candidates and source semantics, making the cost comparison sensitive to the reuse of entry-independent facts. Shared construction produces xxx use-site facts and consumes xxx seconds, compared with xxx seconds for full-body construction. Disabling implicit alternatives changes the number of represented type combinations by xxx. The corresponding differences in paired runtime and annotation-contract detection counts are xxx and xxx. The instance cache serves xxx% of eligible applications. Direct

Table 2. Detection effectiveness. Paired counts and precision (%) are separated by error category. MOPSA uses its supported subset of xxx records, covering xxx% of the full pool.

Method	Runtime		Annotation contract	
	Paired	Precision	Paired	Precision
Mypy	xxx	xxx	xxx	xxx
Pyright	xxx	xxx	xxx	xxx
Pyrefly	xxx	xxx	xxx	xxx
Pyinder	xxx	xxx	xxx	xxx
MOPSA (subset)	xxx	xxx	xxx	xxx
Direct LLM	xxx	xxx	xxx	xxx
Inference + checker	xxx	xxx	xxx	xxx
Our approach	xxx	xxx	xxx	xxx

Table 3. Resolution and cost. U, T, and S denote unresolved, timed-out, and unsupported targets. Time is the median per revision in seconds; total cost includes all runs. Outcome counts follow the aggregation rule in Section 4.1.

Method	Resolved (%)	U	T	S	Time	Total (h)
Mypy	xxx	xxx	xxx	xxx	xxx	xxx
Pyright	xxx	xxx	xxx	xxx	xxx	xxx
Pyrefly	xxx	xxx	xxx	xxx	xxx	xxx
Pyinder	xxx	xxx	xxx	xxx	xxx	xxx
MOPSA (subset)	xxx	xxx	xxx	xxx	xxx	xxx
Direct LLM	xxx	xxx	xxx	xxx	xxx	xxx
Inference + checker	xxx	xxx	xxx	xxx	xxx	xxx
Our approach	xxx	xxx	xxx	xxx	xxx	xxx

body analysis is evaluated with and without instance caching; the corresponding total costs are xxx and xxx hours.

Ordered relations and refinement. The relational-summary comparison preserves intermediate failures and error locations, so it compares how check dependencies are represented and used. It produces xxx differing target decisions and xxx differing unresolved outcomes. The selective and full-callee expansion variants use the same re-inference procedure and contextual information. They expand xxx and xxx source statements per candidate and consume xxx and xxx seconds in conflict analysis. Dependency feedback changes xxx candidates from unresolved to decided, dismisses xxx candidates after relation correction, and establishes xxx runtime and xxx annotation-contract reports. These outcomes are recorded with the responsible summary revision and the affected caller judgment.

4.4 RQ3: New-Error Discovery

The discovery corpus contains xxx repositories independent of the historical benchmark, at revisions xxx. Their selection criteria and analysis entries are xxx. Reports are reviewed using the same category and evidence rules as the historical study. Across these repositories, our approach identifies xxx runtime type errors and xxx annotation-contract errors. Their diagnostic precision is xxx% and xxx%, respectively. The numbers with independent maintainer confirmation are xxx and xxx; the numbers without that confirmation are xxx and xxx.

Table 4. Controlled mechanism comparisons. Each variant changes the named representation or procedure while retaining the common reporting criteria.

Variant	Change
Independent types	Project each case’s location relations onto independent type sets.
Merged paths	Join branch alternatives, removing condition–effect associations.
Unknown effects	Replace potentially written shared-state contents by unknown values.
Full-body construction	Construct relations from the whole function without separate entry-independent fact inference.
No implicit alternatives	Disable additional LLM type-combination proposals while retaining symbolic requirements and repository facts.
Relational summaries	Export intermediate states, normal/failure relations, and error locations; compose by linking intermediate symbols.
Direct body analysis	Analyze callee bodies at calls using the same state and operation rules.
No instance cache	Recompute instantiated results while retaining reusable summary templates.
Full callee expansion	Re-infer complete relevant callee bodies instead of selectively expanded type paths.
No dependency feedback	Assess initial candidates without revising summaries and reanalyzing dependent callers.

Table 5. Mechanism results. Runtime and contract columns report paired detections for their respective error categories. Outcome counts follow the aggregation rule in Section 4.1; total cost includes all runs.

Variant	Runtime	Contract	Unresolved	Total (h)
Complete approach	xxx	xxx	xxx	xxx
Independent types	xxx	xxx	xxx	xxx
Merged paths	xxx	xxx	xxx	xxx
Unknown effects	xxx	xxx	xxx	xxx
Full-body construction	xxx	xxx	xxx	xxx
No implicit alternatives	xxx	xxx	xxx	xxx
Relational summaries	xxx	xxx	xxx	xxx
Direct body analysis	xxx	xxx	xxx	xxx
No instance cache	xxx	xxx	xxx	xxx
Full callee expansion	xxx	xxx	xxx	xxx
No dependency feedback	xxx	xxx	xxx	xxx

For each confirmed error, the evidence record associates the source location with the entry state, relevant calls, conditional updates, and conflicting requirement or declaration. The discovery results contain xxx errors involving attribute or global updates, xxx involving condition–type relations, and xxx involving input/return dependencies. Overlapping mechanism labels remain linked to the same error record. Total analysis time over all runs is xxx hours, including xxx hours of construction and xxx hours of conflict analysis and reanalysis.

5 Threats to Validity

Construct validity. Runtime type errors and annotation-contract errors measure different properties. An open-entry failure can describe an admissible symbolic calling state without establishing a failure reachable from a repository entry. Likewise, an annotation conflict need not cause a runtime exception in an existing call. Our category-specific metrics, declared entries, and cross-category

links preserve these distinctions. Manual judgments about reachability, intended interfaces, duplicate root causes, and exception handling can affect precision. The source and behavioral evidence retained with each report make these judgments inspectable; the adjudication procedure is specified in Section 4.1.

Internal validity. LLM-generated relations can omit legal type behaviors or introduce incorrect ones. Conflict-driven expansion checks disputed relations, but unresolved source semantics and incomplete context can still prevent a decision. The method does not establish soundness or completeness for arbitrary Python programs. The fixed pyflow call graph also bounds the callees available for propagation and expansion; a missing target cannot be recovered by revising a type-state relation alone. Dynamic attribute behavior, reflection, native extensions, unknown dependency effects, and bounded loop or recursive exploration can affect coverage. These cases retain unresolved or unsupported outcomes rather than being treated as confirmed absence of errors.

Ablation differences can arise from semantic coverage or budget consumption as well as the representation being studied. Matched entries, rules, and confirmation criteria help isolate these factors. We retain the causes of undecided targets and compare intermediate failures in the relational and direct-body alternatives. Differences between tools' dependency models and supported Python features remain relevant to the whole-tool comparisons.

External validity. Public GitHub fixes emphasize errors that developers discovered and chose to repair. Repository selection, issue wording, dependency availability, and the required bug-fix pairing can therefore affect the sampled error distribution. The recent collection interval reduces exposure to older public examples but does not establish that the benchmark is absent from model training data. Model training cutoffs and corpus membership may be unknown. Independent discovery repositories test another setting, although their domains and selected entries still bound generalization.

Conclusion validity. Each experimental configuration is run three times, and effectiveness is reported using the best result. The selection metric, aggregation unit, and tie-breaking rule are xxx. This best-of-three protocol reports selected effectiveness rather than expected performance from a single run. It can favor methods with greater run-to-run variability and depends on the selection metric and aggregation unit. Consequently, the reported effectiveness cannot be interpreted as mean reliability. Retaining all run records and accounting for all three runs' cost exposes the associated variability and resource expenditure. The selected run's cost is recorded separately. Model revisions, decoding settings, and external service latency can also change outcomes and cost.

6 Related Work

Python type checking and type-error detection. Python's annotation system supplies optional type declarations, and structural protocols describe behavior shared by different classes [6, 21]. Checkers combine this interface information with inference and local narrowing [12, 18]. Empirical studies examine how gradual typing affects checker behavior and which types are needed to describe dynamic objects [19, 22]. Our analysis focuses on the state at an operation after preceding calls have changed shared locations. The relevant relation connects a calling condition to the particular update that reaches that operation.

Pyinder is a close task-level comparison. It infers types, analyzes functions under parameter-type contexts, and aggregates and filters member types to identify Python type errors [13]. We study a representation that keeps entry state, execution conditions, ordered requirements, and caller-visible updates together. Actual object bindings instantiate this representation at calls, while conflict

dependencies identify the callee type paths used for re-inference. The whole-tool comparison measures detection outcomes; the controlled variants examine the roles of relation preservation and feedback.

Type inference and generated annotations. Annotation inference reconstructs interface types from code and repository context. PyTIR models entity dependencies and iteratively updates types and dependencies with checker feedback [20]. Its evaluation includes annotation accuracy and diagnostics introduced by generated annotations. Our target is the original program's operation behavior and existing annotation contracts. The state analysis uses a fixed pyflow call graph [4]; refinement changes type-state relations and their dependent judgments. The inference-plus-checker baseline evaluates how annotation generation serves this detection task, separating original errors from generated-interface inconsistencies.

Relational and compositional program analysis. Hoare logic connects assertions at program entry and exit [5]. Compositional symbolic execution reuses procedure behavior at calls [3], while abstract interpretation provides a foundation for reasoning about program states and their relations [1]. MOPSA applies abstract interpretation to Python type analysis, including state relations and control and exception behavior [10].

Our model uses these relational and compositional principles for Python type-error detection. Its cases preserve ordered checks and writes along with entry and exit assertions, so intermediate failure dependencies remain available during composition and refinement. The accompanying construction procedure shares entry-independent facts and proposes implicit type alternatives; detection revisits disputed alternatives through selectively expanded type paths. The comparison with failure-preserving relational summaries and direct body analysis evaluates these representation and construction choices under matched rules.

7 Conclusion

We present an interprocedural type-state analysis for detecting Python type errors. The model connects function inputs and execution conditions to ordered operation requirements, shared-state updates, and exits. The analysis constructs and composes these relations, then uses conflicts to select callee type paths for expansion and contextual re-inference. Dependency-driven revisions update affected caller judgments. Runtime type errors and annotation-contract errors retain distinct confirmation criteria and results. The evaluation measures paired detection, diagnostic precision, unresolved outcomes, and complete analysis cost. The central design connects each error judgment to the state established by the calls and updates preceding the operation.

References

- [1] Patrick Cousot and Radhia Cousot. 1977. Abstract Interpretation: A Unified Lattice Model for Static Analysis of Programs by Construction or Approximation of Fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN Symposium on Principles of Programming Languages (POPL '77)*. ACM, New York, NY, USA, 238–252. <https://www.di.ens.fr/~cousot/COUSOTpapers/POPL77.shtml>
- [2] Dropbox. 2019. Our Journey to Type Checking 4 Million Lines of Python. Retrieved September 22, 2026 from <https://dropbox.tech/application/our-journey-to-type-checking-4-million-lines-of-python>
- [3] Patrice Godefroid. 2007. Compositional Dynamic Test Generation. In *Proceedings of the 34th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '07)*. Association for Computing Machinery, New York, NY, USA, 47–54. https://patricegodefroid.github.io/public_psfiles/popl2007.pdf
- [4] Zinan Gu, Haoxiang Yan, and Peisen Yao. 2026. PyFlow: An Inter-procedural Static Analysis Framework for Python. arXiv:2608.07026 doi:10.48550/arXiv.2608.07026
- [5] C. A. R. Hoare. 1969. An Axiomatic Basis for Computer Programming. *Commun. ACM* 12, 10 (1969), 576–580, 583. doi:10.1145/363235.363259

- [6] Ivan Levkivskiy, Jukka Lehtosalo, and Łukasz Langa. 2017. PEP 544 – Protocols: Structural Subtyping (Static Duck Typing). Retrieved September 9, 2026 from <https://peps.python.org/pep-0544/>
- [7] Meta Platforms, Inc. n.d.. Pyreify. Retrieved September 22, 2026 from <https://github.com/facebook/pyreify>
- [8] Meta Platforms, Inc. n.d.. Pyreify: A Fast Python Type Checker and Language Server. Retrieved September 22, 2026 from <https://pyreify.org/>
- [9] Microsoft. n.d.. Pyright: Static Type Checker for Python. Retrieved September 22, 2026 from <https://github.com/microsoft/pyright>
- [10] Raphaël Monat, Abdelraouf Ouadjaout, and Antoine Miné. 2020. Static Type Analysis by Abstract Interpretation of Python Programs. In *34th European Conference on Object-Oriented Programming (Leibniz International Proceedings in Informatics, Vol. 166)*. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 17:1–17:29. doi:10.4230/LIPIcs.ECOOP.2020.17
- [11] Mypy contributors. n.d.. The mypy Command Line. Mypy Documentation. Retrieved September 22, 2026 from https://mypy.readthedocs.io/en/stable/command_line.html
- [12] Mypy contributors. n.d.. Type Narrowing. Mypy Documentation. Retrieved September 16, 2026 from https://mypy.readthedocs.io/en/stable/type_narrowing.html
- [13] Wonseok Oh and Hakjoo Oh. 2024. Towards Effective Static Type-Error Detection for Python. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering (ASE '24)*. Association for Computing Machinery, 1808–1820. doi:10.1145/3691620.3695545
- [14] Benjamin C. Pierce, Arthur Azevedo de Amorim, Chris Casinghino, Marco Gaboardi, Michael Greenberg, Cătălin Hrițcu, Vilhelm Sjöberg, Andrew Tolmach, and Brent Yorgey. 2026. Hoare Logic, Part I. Software Foundations, Volume 2: Programming Language Foundations. Retrieved September 16, 2026 from <https://softwarefoundations.cis.upenn.edu/plf-current/Hoare.html> Version 7.1.
- [15] Python Software Foundation. n.d.. Data Model. Python Language Reference. Retrieved September 15, 2026 from <https://docs.python.org/3/reference/datamodel.html>
- [16] Python Software Foundation. n.d.. Execution Model. Python Language Reference. Retrieved September 16, 2026 from <https://docs.python.org/3/reference/executionmodel.html>
- [17] Python Software Foundation. n.d.. Glossary: Duck-Typing. Python Documentation. Retrieved September 16, 2026 from <https://docs.python.org/3/glossary.html#term-duck-typing>
- [18] Python typing contributors. n.d.. Type Narrowing. Python Typing Documentation. Retrieved September 16, 2026 from https://typing.python.org/en/latest/guides/type_narrowing.html
- [19] Ke Sun, Sheng Chen, Meng Wang, and Dan Hao. 2023. What Types Are Needed for Typing Dynamic Objects? A Python-Based Empirical Study. In *Programming Languages and Systems*. Springer Nature Singapore, 24–45. doi:10.1007/978-981-99-8311-7_2
- [20] Shuo Sun, Shixin Zhang, Jiwei Yan, Jun Yan, and Jian Zhang. 2025. Co-Evolution of Types and Dependencies: Towards Repository-Level Type Inference for Python Code. arXiv:2512.21591 [cs.SE] doi:10.48550/arXiv.2512.21591
- [21] Guido van Rossum, Jukka Lehtosalo, and Łukasz Langa. 2014. PEP 484 – Type Hints. Retrieved September 9, 2026 from <https://peps.python.org/pep-0484/>
- [22] Wenjie Xu, Lin Chen, Chenghao Su, Yimeng Guo, Yanhui Li, Yuming Zhou, and Baowen Xu. 2023. How Well Static Type Checkers Work with Gradual Typing? A Case Study on Python. In *Proceedings of the 31st IEEE/ACM International Conference on Program Comprehension (ICPC '23)*. IEEE, 242–253. doi:10.1109/ICPC58990.2023.00039